

TASK 7

We can now create a small SQLite database file named sales_data.db with a single table called sales. By using the sqlite3 module to generate this file and insert a few sample records.

This project centers around a structured sales dataset stored in a SQLite database (sales_data.db). The dataset captures transactional information, including the date of sale, customer name, product sold, quantity, and unit price. It is designed to support exploratory data analysis, business intelligence reporting, and testing of SQL and data science skills.

The dataset can be used for:

- Tracking sales performance over time
- Analyzing customer purchasing patterns
- Calculating revenue and identifying top-selling products
- Building dashboards or predictive models for sales forecasting

```
import sqlite3
# Connect to SQLite database (it will create the file if it doesn't exist)
conn = sqlite3.connect("sales_data.db")
cursor = conn.cursor()
# Create the sales table
cursor.execute("""
CREATE TABLE IF NOT EXISTS sales (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    date TEXT NOT NULL,
    customer TEXT NOT NULL,
    product TEXT NOT NULL,
    quantity INTEGER NOT NULL,
    price REAL NOT NULL
)
""")
```

<sqlite3.Cursor at 0x78eb25d59d40>

```
# Insert some sample data
sample_data = [
    ("2025-04-01", "Alice Smith", "Laptop", 1, 1200.00),
    ("2025-04-02", "Bob Johnson", "Smartphone", 2, 800.00),
    ("2025-04-03", "Charlie Lee", "Headphones", 3, 150.00),
    ("2025-04-04", "Dana Kim", "Monitor", 1, 300.00),
```

```
( "2025-04-05", "Eli Zhao", "Keyboard", 2, 100.00),  
( "2025-04-06", "Don Regin", "Laptop", 3, 250.00),  
( "2025-04-07", "Frank Wong", "Smartphone", 1, 900.00),  
( "2025-04-08", "Gina Chen", "Headphones", 2, 120.00),  
( "2025-04-09", "Leona Dsouza", "Keyboard", 3, 345.00),  
( "2025-04-10", "Ferald Sold", "Headphones", 1, 500.00),  
( "2025-04-11", "Fwadert Mold", "Headphones", 2, 600.00),  
( "2025-04-12", "Franklin John", "Monitor", 3, 550.00)  
]
```

```
cursor.executemany("INSERT INTO sales (date, customer, product,  
quantity, price) VALUES (?, ?, ?, ?, ?)", sample_data)
```

```
<sqlite3.Cursor at 0x78eb25d59d40>
```

```
# Commit changes and close the connection  
conn.commit()  
conn.close()
```

This will create a table named sales with the following columns:

- id: unique ID for each record
- date: date of the sale
- customer : name of the customer
- product: product name
- quantity: quantity sold
- price: price per unit

To load and connect to the sales_data.db SQLite database using Python, you just need to make sure you're referencing the correct path to the file if it's not in your current working directory.

Here's the full version using the correct path.

```
import sqlite3  
# Connect to the SQLite database  
conn = sqlite3.connect("sales_data.db") # if it's in the same  
directory  
# Create a cursor object  
cursor = conn.cursor()  
# Example: Fetch and print all sales records
```

```
cursor.execute("SELECT * FROM sales")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

```
(1, '2025-04-01', 'Alice Smith', 'Laptop', 1, 1200.0)
(2, '2025-04-02', 'Bob Johnson', 'Smartphone', 2, 800.0)
(3, '2025-04-03', 'Charlie Lee', 'Headphones', 3, 150.0)
(4, '2025-04-04', 'Dana Kim', 'Monitor', 1, 300.0)
(5, '2025-04-05', 'Eli Zhao', 'Keyboard', 2, 100.0)
(6, '2025-04-06', 'Don Regin', 'Laptop', 3, 250.0)
(7, '2025-04-07', 'Frank Wong', 'Smartphone', 1, 900.0)
(8, '2025-04-08', 'Gina Chen', 'Headphones', 2, 120.0)
(9, '2025-04-09', 'Leona Dsouza', 'Keyboard', 3, 345.0)
(10, '2025-04-10', 'Ferald Sold', 'Headphones', 1, 500.0)
(11, '2025-04-11', 'Fwadert Mold', 'Headphones', 2, 600.0)
(12, '2025-04-12', 'Franklin John', 'Monitor', 3, 550.0)
```

Here are a few examples of what you can do after loading the database:

1. View All Records

```
import sqlite3
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
# Example: Fetch and print all sales records
cursor.execute("SELECT * FROM sales")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

```
(1, '2025-04-01', 'Alice Smith', 'Laptop', 1, 1200.0)
(2, '2025-04-02', 'Bob Johnson', 'Smartphone', 2, 800.0)
(3, '2025-04-03', 'Charlie Lee', 'Headphones', 3, 150.0)
(4, '2025-04-04', 'Dana Kim', 'Monitor', 1, 300.0)
(5, '2025-04-05', 'Eli Zhao', 'Keyboard', 2, 100.0)
(6, '2025-04-06', 'Don Regin', 'Laptop', 3, 250.0)
(7, '2025-04-07', 'Frank Wong', 'Smartphone', 1, 900.0)
(8, '2025-04-08', 'Gina Chen', 'Headphones', 2, 120.0)
(9, '2025-04-09', 'Leona Dsouza', 'Keyboard', 3, 345.0)
(10, '2025-04-10', 'Ferald Sold', 'Headphones', 1, 500.0)
(11, '2025-04-11', 'Fwadert Mold', 'Headphones', 2, 600.0)
(12, '2025-04-12', 'Franklin John', 'Monitor', 3, 550.0)
```

2. Filter Sales by Customer

```
import sqlite3
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
cursor.execute("SELECT * FROM sales WHERE customer = 'Alice Smith'")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

```
(1, '2025-04-01', 'Alice Smith', 'Laptop', 1, 1200.0)
```

3. Calculate Total Revenue

```
import sqlite3
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
cursor.execute("SELECT SUM(quantity * price) FROM sales")
total_revenue = cursor.fetchone()[0]
print("Total Revenue:", total_revenue)
```

```
Total Revenue: 10025.0
```

4. Group Sales by Product

```
import sqlite3
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
cursor.execute("""
    SELECT product, SUM(quantity) AS total_quantity, SUM(quantity *
price) AS total_revenue
    FROM sales
    GROUP BY product
""")
rows = cursor.fetchall()
```

```
for row in rows:
    print(row)
```

```
('Headphones', 8, 2390.0)
('Keyboard', 5, 1235.0)
('Laptop', 4, 1950.0)
('Monitor', 4, 1950.0)
('Smartphone', 3, 2500.0)
```

Here's how you can run that basic SQL query in Python and print the results:

```
import sqlite3
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
# SQL query to group sales by product
query = """
SELECT
    product,
    SUM(quantity) AS total_qty,
    SUM(quantity * price) AS revenue
FROM sales
GROUP BY product
"""
# Execute and fetch results
cursor.execute(query)
results = cursor.fetchall()
# Print results
print("Product | Total Quantity | Revenue")
for row in results:
    print(f"{row[0]} | {row[1]} | {row[2]:.2f}")
```

```
Product | Total Quantity | Revenue
Headphones | 8 | 2390.00
Keyboard | 5 | 1235.00
Laptop | 4 | 1950.00
Monitor | 4 | 1950.00
Smartphone | 3 | 2500.00
```

This will output a summary of sales for each product, showing total quantity sold and total revenue generated.

```
import sqlite3
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
```

```

# Create a cursor object
cursor = conn.cursor()
# SQL query to group sales by product
query = """
SELECT
    product,
    SUM(quantity) AS total_qty,
    SUM(quantity * price) AS revenue
FROM sales
GROUP BY product
"""
# Load query result into a pandas DataFrame
df = pd.read_sql_query(query, conn)

# Display the DataFrame
print(df)

```

	product	total_qty	revenue
0	Headphones	8	2390.0
1	Keyboard	5	1235.0
2	Laptop	4	1950.0
3	Monitor	4	1950.0
4	Smartphone	3	2500.0

This will give you a nicely formatted table in df that you can use for analysis or visualization.

Here's how you can plot the sales data by product using matplotlib:

Total Quantity and Revenue by Product

```

import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
# SQL query to group sales by product
query = """
SELECT
    product,
    SUM(quantity) AS total_qty,
    SUM(quantity * price) AS revenue
FROM sales
GROUP BY product
"""

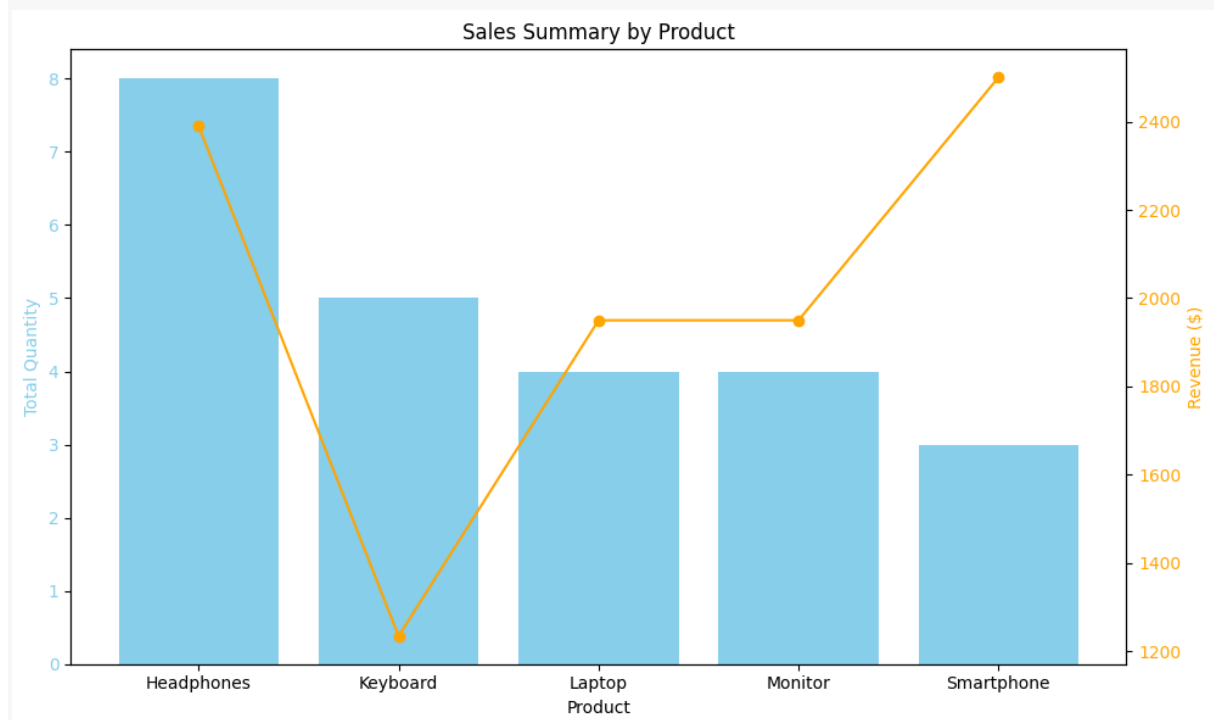
```

```

# Load query result into a pandas DataFrame
df = pd.read_sql_query(query, conn)
# Plotting
fig, ax1 = plt.subplots(figsize=(10, 6))
# Bar chart for quantity
ax1.bar(df['product'], df['total_qty'], color='skyblue', label='Total
Quantity')
ax1.set_ylabel('Total Quantity', color='skyblue')
ax1.set_xlabel('Product')
ax1.tick_params(axis='y', labelcolor='skyblue')
# Twin axis for revenue
ax2 = ax1.twinx()
ax2.plot(df['product'], df['revenue'], color='orange', marker='o',
label='Revenue')
ax2.set_ylabel('Revenue ($)', color='orange')
ax2.tick_params(axis='y', labelcolor='orange')

plt.title('Sales Summary by Product')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



This creates a bar + line combo plot:

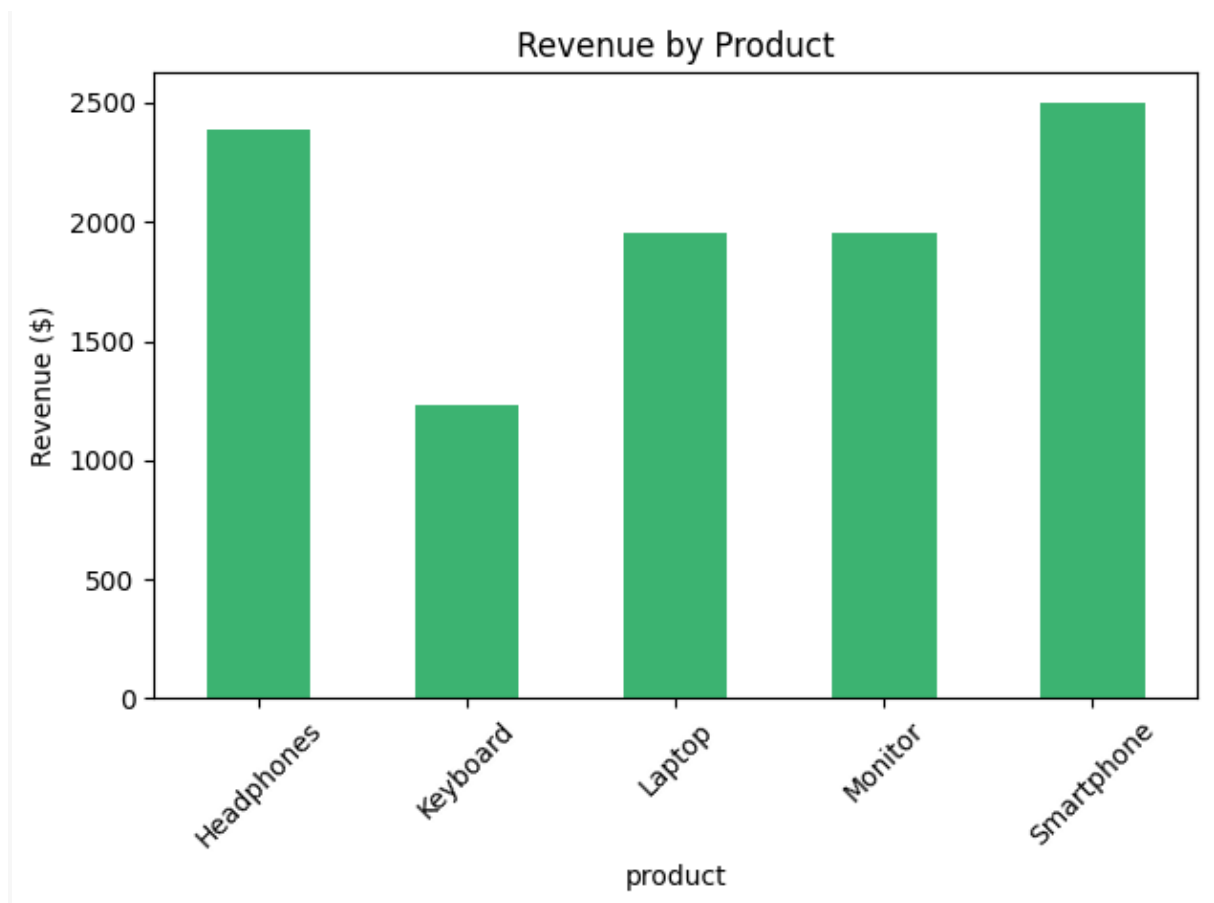
- Blue bars = total quantity sold
- Orange line = total revenue per product

Here's how you can plot a simple bar chart using just pandas and matplotlib to show revenue by product:

Simple Bar Chart (Revenue by Product)

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
# SQL query to group sales by product
query = """
SELECT
    product,
    SUM(quantity) AS total_qty,
    SUM(quantity * price) AS revenue
FROM sales
GROUP BY product
"""
# Load query result into a pandas DataFrame
df = pd.read_sql_query(query, conn)
# Simple bar chart
df.plot(kind='bar', x='product', y='revenue', legend=False,
color='mediumseagreen')

# Add labels and title
plt.ylabel('Revenue ($)')
plt.title('Revenue by Product')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



This will display a clean vertical bar chart of revenue by product.

Here's how you can plot both revenue and total quantity side by side for each product using a grouped bar chart:

Grouped Bar Chart: Revenue & Quantity by Product

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
# SQL query to group sales by product
```

```

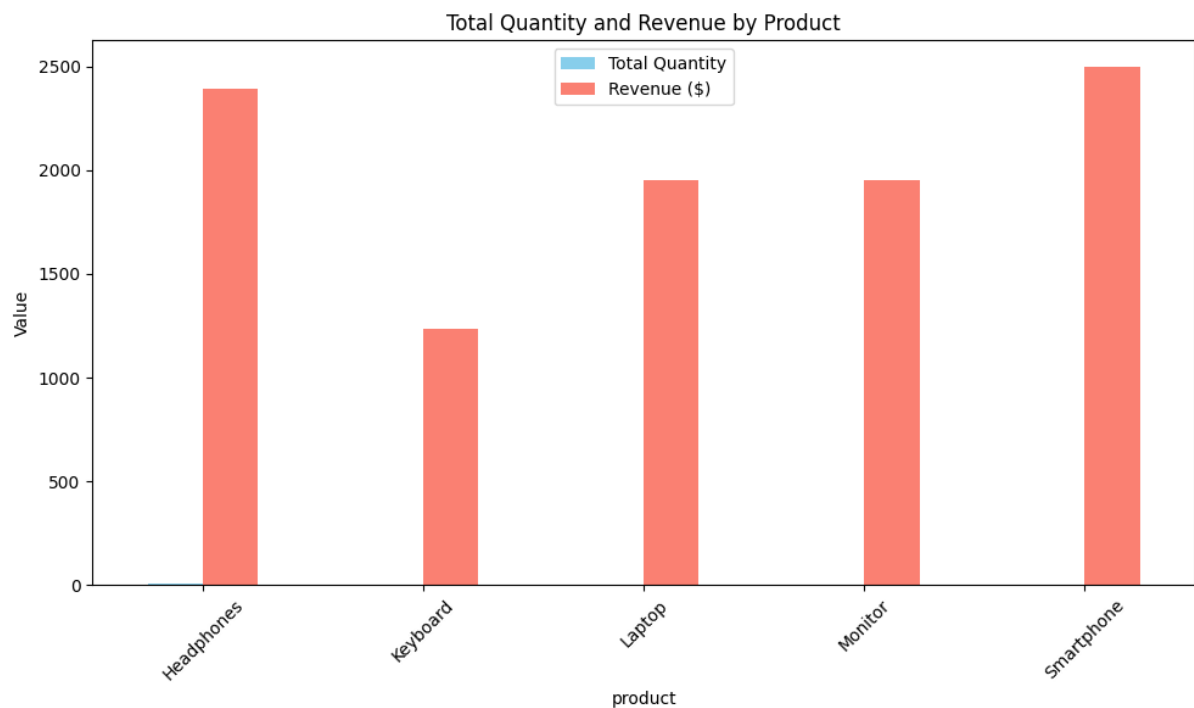
query = """
SELECT
    product,
    SUM(quantity) AS total_qty,
    SUM(quantity * price) AS revenue
FROM sales
GROUP BY product
"""

# Load query result into a pandas DataFrame
df = pd.read_sql_query(query, conn)
# Set the product column as index for better plotting
df.set_index('product', inplace=True)

# Plot both quantity and revenue
df[['total_qty', 'revenue']].plot(kind='bar', figsize=(10, 6),
color=['skyblue', 'salmon'])

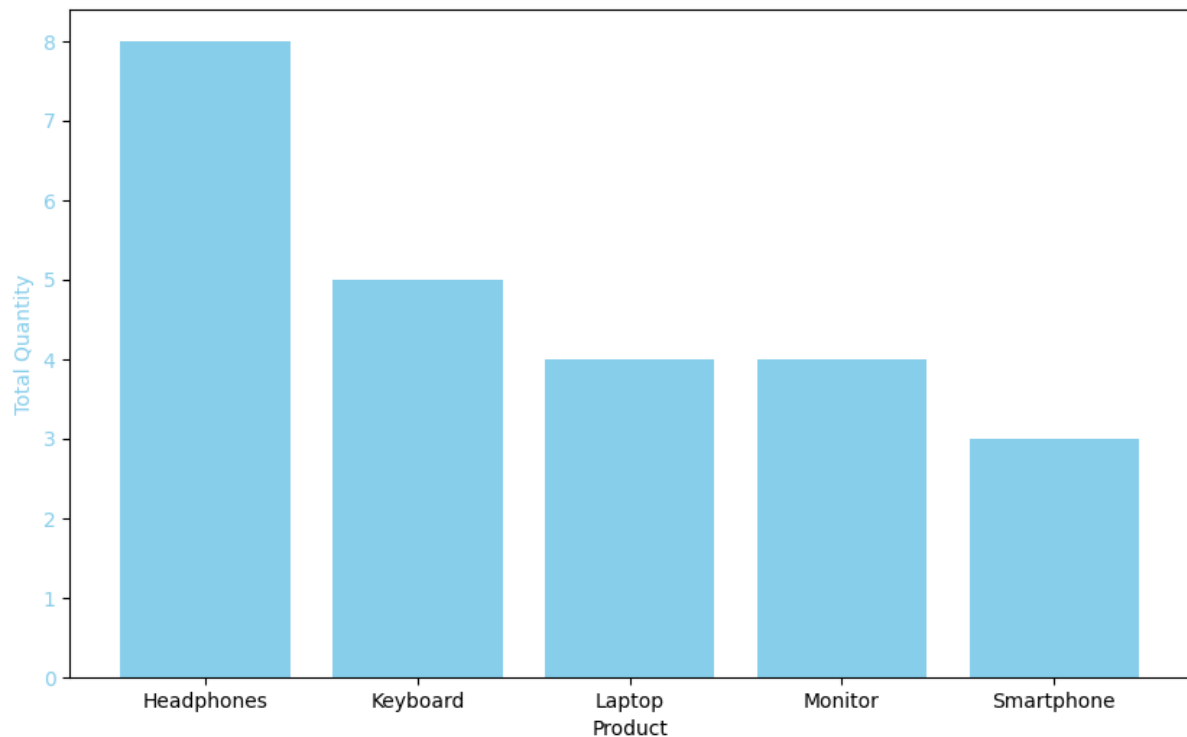
# Add labels and title
plt.ylabel('Value')
plt.title('Total Quantity and Revenue by Product')
plt.xticks(rotation=45)
plt.tight_layout()
plt.legend(['Total Quantity', 'Revenue ($)'])
plt.show()

```



This will show a grouped bar chart, where each product has two bars: one for quantity and one for revenue.

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
# SQL query to group sales by product
query = """
SELECT
    product,
    SUM(quantity) AS total_qty,
    SUM(quantity * price) AS revenue
FROM sales
GROUP BY product
"""
# Load query result into a pandas DataFrame
df = pd.read_sql_query(query, conn)
# Plotting
fig, ax1 = plt.subplots(figsize=(10, 6))
# Bar chart for quantity
ax1.bar(df['product'], df['total_qty'], color='skyblue', label='Total
Quantity')
ax1.set_ylabel('Total Quantity', color='skyblue')
ax1.set_xlabel('Product')
ax1.tick_params(axis='y', labelcolor='skyblue')
```



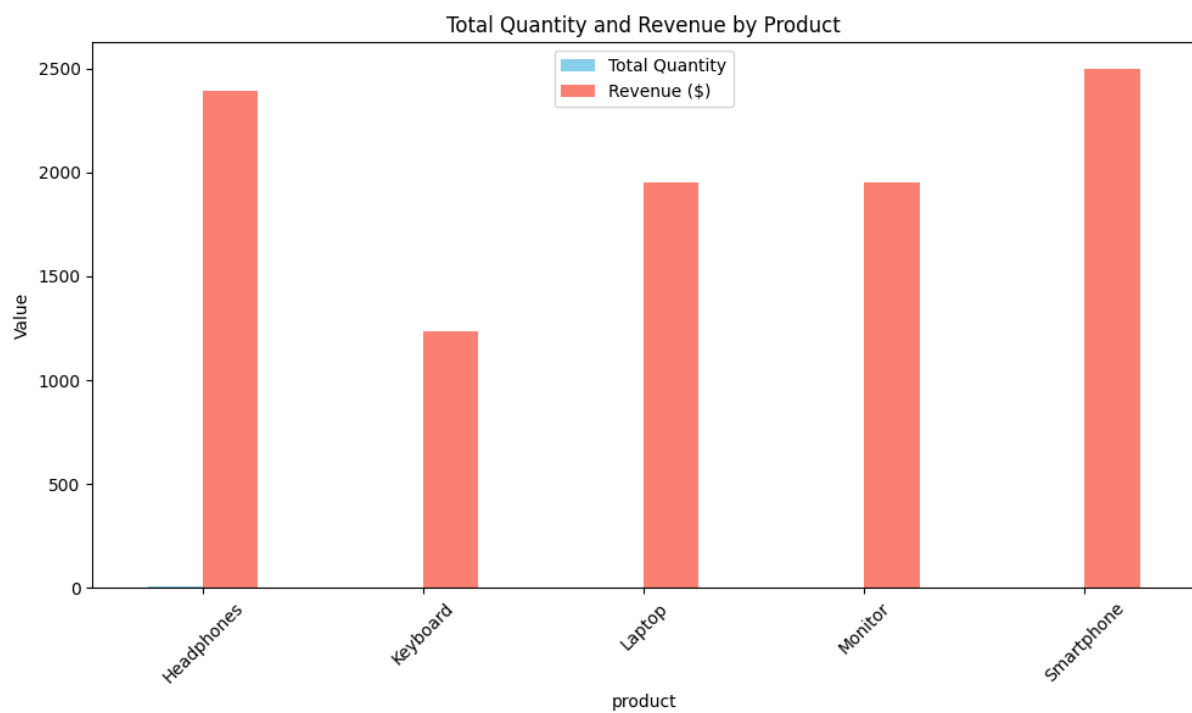
```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db") # if it's in the same
directory
# Create a cursor object
cursor = conn.cursor()
# SQL query to group sales by product
query = """
SELECT
    product,
    SUM(quantity) AS total_qty,
    SUM(quantity * price) AS revenue
FROM sales
GROUP BY product
"""
# Load query result into a pandas DataFrame
df = pd.read_sql_query(query, conn)

# Set index (if not already done)
df.set_index('product', inplace=True)

# Plot both columns
```

```
df[['total_qty', 'revenue']].plot(kind='bar', figsize=(10, 6),
color=['skyblue', 'salmon'])

# Final touches
plt.ylabel('Value')
plt.title('Total Quantity and Revenue by Product')
plt.xticks(rotation=45)
plt.tight_layout()
plt.legend(['Total Quantity', 'Revenue ($)'])
plt.show()
```



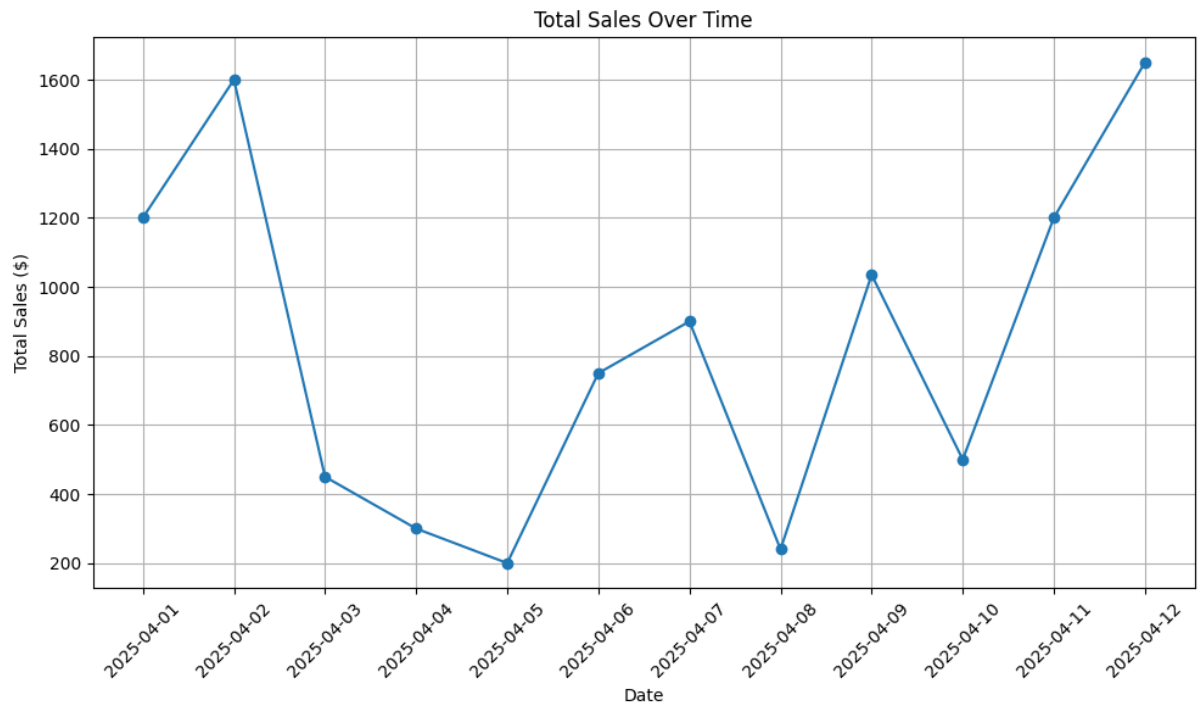
Line chart of total sales over time

Here's a full example using Matplotlib to visualize sales over time from the sales table:

```
# Connect to the database
conn = sqlite3.connect("sales_data.db")

# Load data into a pandas DataFrame
query = """
SELECT date, SUM(quantity * price) as total_sales
FROM sales
GROUP BY date
ORDER BY date
"""
df = pd.read_sql_query(query, conn)

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(df['date'], df['total_sales'], marker='o')
plt.title("Total Sales Over Time")
plt.xlabel("Date")
plt.ylabel("Total Sales ($)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.show()
```

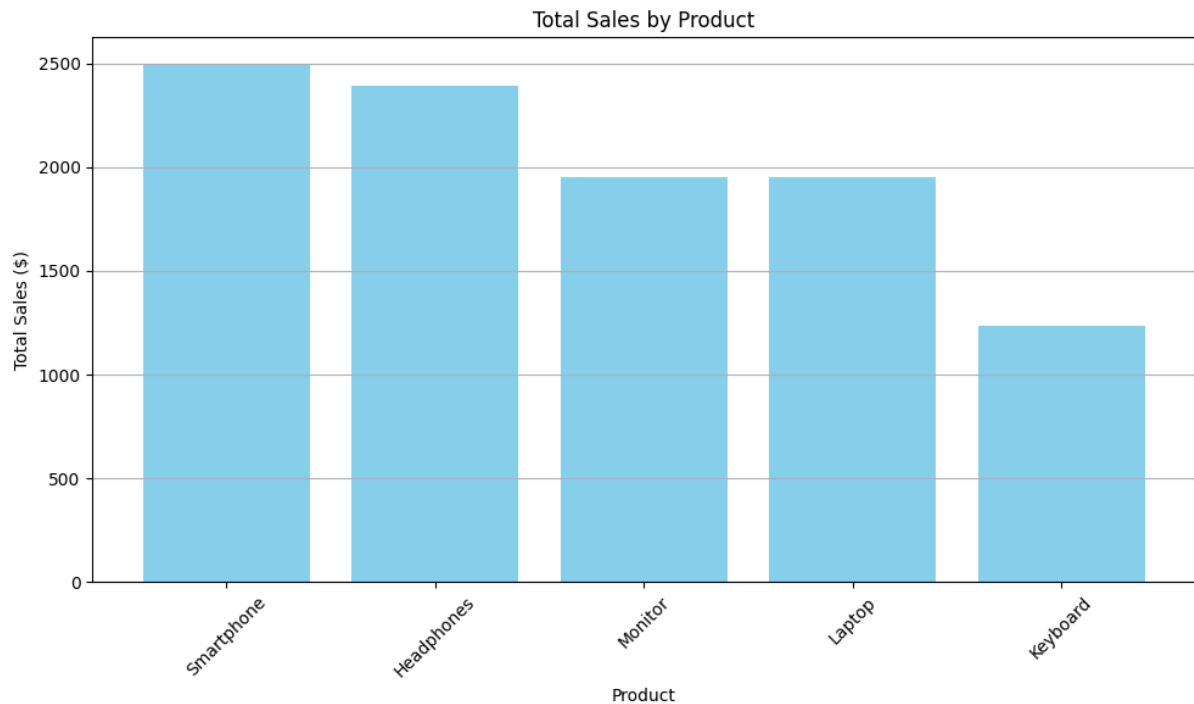


Bar chart — Total Sales by Product

```
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")

# Query: total sales by product
query = """
SELECT product, SUM(quantity * price) AS total_sales
FROM sales
GROUP BY product
ORDER BY total_sales DESC
"""
df = pd.read_sql_query(query, conn)

# Plotting the bar chart
plt.figure(figsize=(10, 6))
plt.bar(df['product'], df['total_sales'], color='skyblue')
plt.title("Total Sales by Product")
plt.xlabel("Product")
plt.ylabel("Total Sales ($)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(axis='y')
plt.show()
```



This chart helps you identify which products are generating the most revenue. If your dataset is large, I can also show you how to display only the top 5 or top 10 products.

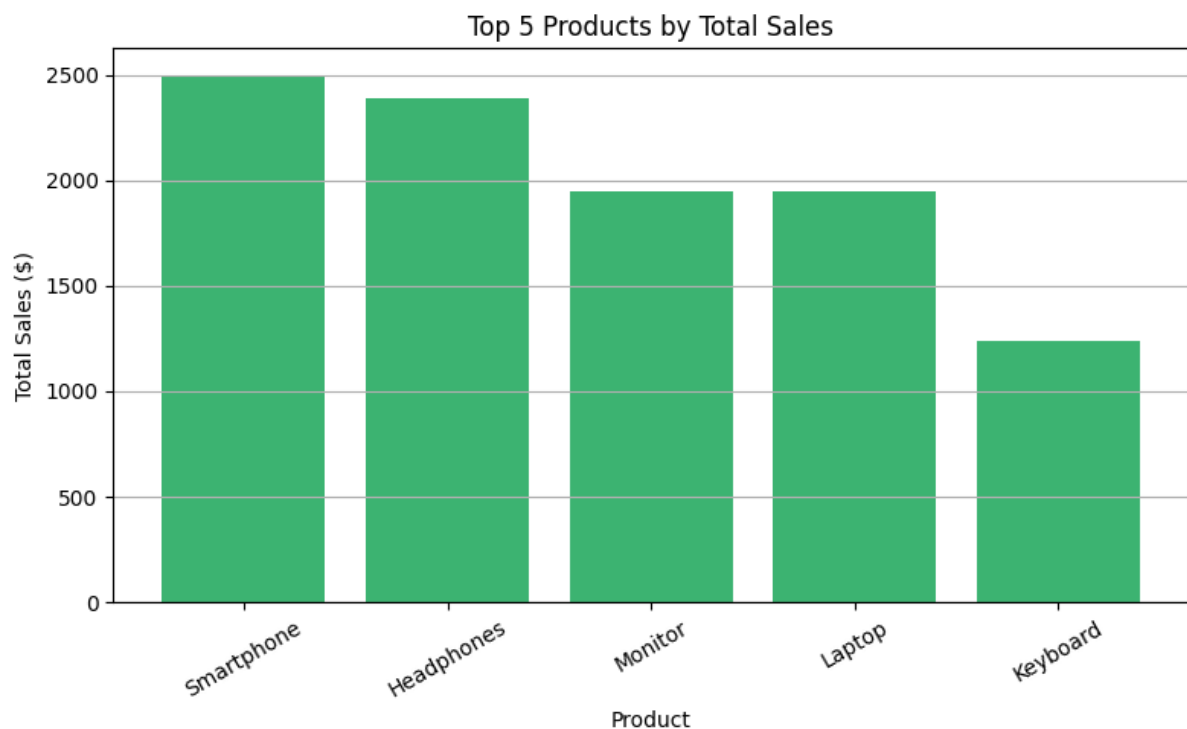
Here's how to modify the previous bar chart to show only the top 5 products by total sales:

Bar Chart: Top 5 Products by Total Sales

```
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")

# Query: top 5 products by total sales
query = """
SELECT product, SUM(quantity * price) AS total_sales
FROM sales
GROUP BY product
ORDER BY total_sales DESC
LIMIT 5
"""
df = pd.read_sql_query(query, conn)
```

```
# Plotting the bar chart
plt.figure(figsize=(8, 5))
plt.bar(df['product'], df['total_sales'], color='mediumseagreen')
plt.title("Top 5 Products by Total Sales")
plt.xlabel("Product")
plt.ylabel("Total Sales ($)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.grid(axis='y')
plt.show()
```



This will give you a clean view of your best performers in terms of revenue.

Here's a modified version of the chart that shows the Top 5 Products by Quantity Sold instead of revenue:

Bar Chart: Top 5 Products by Quantity Sold

```
# Connect to the SQLite database
conn = sqlite3.connect("sales_data.db")

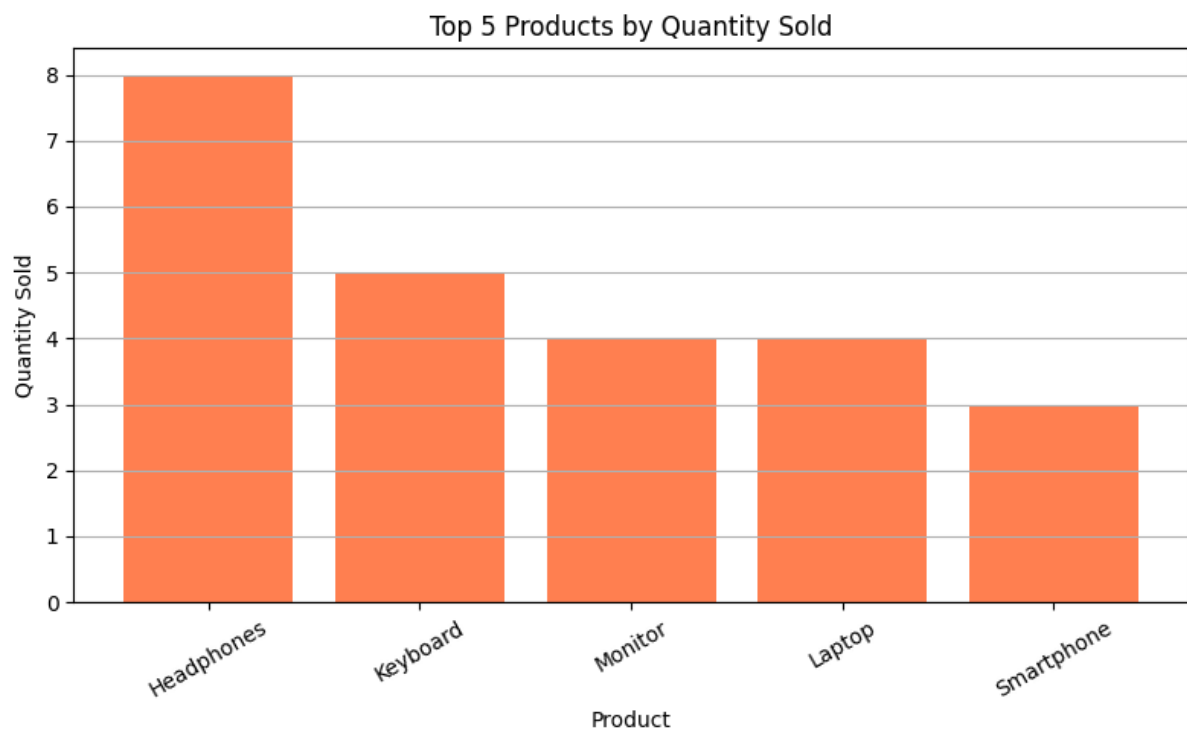
# Query: top 5 products by quantity sold
query = """
SELECT product, SUM(quantity) AS total_quantity
FROM sales
GROUP BY product
ORDER BY total_quantity DESC
```

```

LIMIT 5
"""
df = pd.read_sql_query(query, conn)

# Plotting the bar chart
plt.figure(figsize=(8, 5))
plt.bar(df['product'], df['total_quantity'], color='coral')
plt.title("Top 5 Products by Quantity Sold")
plt.xlabel("Product")
plt.ylabel("Quantity Sold")
plt.xticks(rotation=30)
plt.tight_layout()
plt.grid(axis='y')
plt.show()

```



This chart will help you see which products are the most frequently purchased, regardless of price.

Here's how you can combine both revenue and quantity sold in a single chart using a dual-axis bar/line chart.

Combo Chart: Top 5 Products by Quantity and Revenue

```
conn = sqlite3.connect("sales_data.db")

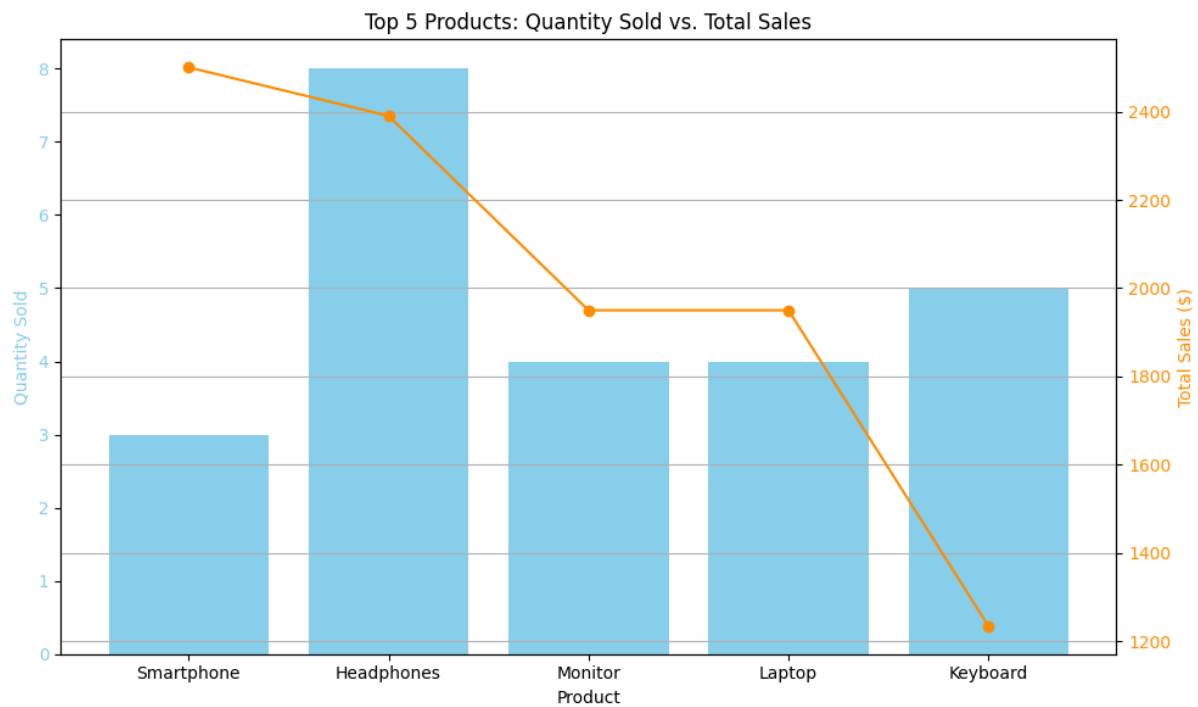
# Query: get total quantity and total sales per product
query = """
SELECT product,
       SUM(quantity) AS total_quantity,
       SUM(quantity * price) AS total_sales
FROM sales
GROUP BY product
ORDER BY total_sales DESC
LIMIT 5
"""
df = pd.read_sql_query(query, conn)

# Plotting
fig, ax1 = plt.subplots(figsize=(10, 6))

# Bar chart for quantity sold
ax1.bar(df['product'], df['total_quantity'], color='skyblue',
        label='Quantity Sold')
ax1.set_xlabel('Product')
ax1.set_ylabel('Quantity Sold', color='skyblue')
ax1.tick_params(axis='y', labelcolor='skyblue')

# Create a second y-axis for total sales
ax2 = ax1.twinx()
ax2.plot(df['product'], df['total_sales'], color='darkorange',
        marker='o', label='Total Sales ($)')
ax2.set_ylabel('Total Sales ($)', color='darkorange')
ax2.tick_params(axis='y', labelcolor='darkorange')

# Title and layout
plt.title('Top 5 Products: Quantity Sold vs. Total Sales')
fig.tight_layout()
plt.grid(True)
plt.show()
```



This shows:

- Bars represent quantity sold.
- Line represents total sales revenue for the same products.

This is useful to spot high-volume low-value vs. low-volume high-value products.

To save the chart as an image file, just add the `plt.savefig()` line before `plt.show()` like this:

```
plt.savefig("sales_chart.png") # Save the chart as a PNG image
plt.show() # Then display it
```

<Figure size 640x480 with 0 Axes>